RACS — Phase-by-Phase Implementation & Learning Plan

Goal

This plan converts the RACS Final Complete Project Overview into an implementation roadmap tailored to your current level.

You already have good Python, ML, data-processing, SQL, web-development, and project-building experience. The main gaps for RACS are diffusion models, latent diffusion/VAE, CLIP/FAISS, EDICT/invertible diffusion, VLM-based ranking, image-to-image editing, semantic communication, and steganalysis.

The key rule is:

> ***Do not build the whole system at once. Learn one component, build a small version, test it, and then integrate it.***

---

1. The Project in One Picture

Original AgentSemSteCom


Secret Image
↓
Semantic Extraction
↓
Structural Feature
↓
Diffusion Reference Generation
↓
EDICT Steganography
↓
Semantic Communication
↓
Wireless Channel
↓
EDICT Recovery
↓
Recovered Secret

RACS

```
Secret Image
    ↓
Semantic + Structural Extraction
    ↓
CLIP Query
    ↓
FAISS Retrieval
    ↓
Structure-Aware Reranking
    ↓
Token-Controlled Selection
    ↓
Optional Lightweight Editing
    ↓
EDICT Steganography
    ↓
Semantic Communication
    ↓
Wireless Channel
    ↓
Same Retrieval + Same Token
    ↓
EDICT Recovery
    ↓
Recovered Secret
```

The **primary initial change** is:

```
Original: generate reference image
RACS:     retrieve reference image
```

Everything downstream should remain as unchanged as possible so that the experiment is scientifically interpretable.

The overview explicitly defines the transmitter/receiver consistency requirement: database, embedding model, preprocessing, FAISS index, ranking, Top-K, token mapping, editing parameters, and EDICT parameters must agree.

□filecite□turn14file1□

---

2. The Correct Learning Strategy

Do not begin with:

```
LLM + VLM + CLIP + FAISS + Diffusion + EDICT + JSCC
```

Instead:

```
Phase 1  Environment
Phase 2  Image/latent/VAE fundamentals
Phase 3  Diffusion fundamentals
Phase 4  Retrieval database
Phase 5  CLIP embeddings
Phase 6  FAISS retrieval
Phase 7  Token-controlled deterministic selection
Phase 8  Structural representation
Phase 9  Structural reranking
Phase 10 EDICT theory + reproduction
Phase 11 EDICT steganography core
Phase 12 RACS + EDICT
Phase 13 AWGN
Phase 14 Semantic communication
Phase 15 Full RACS-Base
Phase 16 VLM/agentic reranking
Phase 17 Lightweight editing
Phase 18 Retrieve-or-generate fallback
Phase 19 Security/steganalysis
Phase 20 Full experiments + ablations
Phase 21 Final analysis/paper
```

---

3. Phase 1 — Environment and Project Skeleton

Objective

Create a reproducible environment before touching the algorithm.

Learn

- Python virtual environments
- PyTorch
- CUDA/GPU
- Hugging Face model loading
- checkpoints
- tensors
- inference vs training
- reproducibility/random seeds

Build

```
RACS/
├── configs/
├── data/
├── models/
├── semantic/
├── retrieval/
├── reference/
├── steganography/
├── communication/
├── security/
├── evaluation/
├── experiments/
└── scripts/
```

Create one config containing:

```
image_size
device
seed
database_path
index_path
top_k
token
snr
```

First experiment

```
Image
 ↓
Tensor
 ↓
GPU
 ↓
Small pretrained model
 ↓
Output
```

Completion criterion

You can explain:

- image tensors,
- batch/channel dimensions,
- CPU vs GPU,
- model checkpoints,
- why seeds matter.

---

4. Phase 2 — Image Tensors, VAE and Latent Space

Why this phase matters

EDICT and diffusion operate in latent space. You should understand:

```
Image
 ↓
VAE Encoder
 ↓
Latent
 ↓
VAE Decoder
 ↓
Image
```

before trying to understand EDICT.

Learn

- `B × C × H × W`
- normalization
- encoder/decoder
- latent representation
- compression
- pixel space vs latent space

Build

Take one image and run:

```
image
 ↓
VAE encode
 ↓
latent
 ↓
VAE decode
 ↓
reconstructed image
```

Measure:

```
MSE
PSNR
SSIM
```

You should be able to answer

> *Why does Stable Diffusion work in latent space instead of directly in pixel space?*

---

5. Phase 3 — Diffusion Models

Objective

Understand what RACS is trying to replace.

Learn

Conceptually:

```
Forward:

clean image
 ↓
add noise
 ↓
more noise
 ↓
noise
```

and:

```
Reverse:

noise
 ↓
denoise
 ↓
denoise
 ↓
image
```

Then learn:

- timestep
- noise schedule
- UNet

- conditioning
- latent diffusion
- DDIM
- inference steps
- classifier-free guidance

Build

Run a pretrained Stable Diffusion pipeline.

Experiment with:

```
seed
prompt
number of steps
guidance
generation time
GPU memory
```

Key insight

A reference image is expensive because generation performs repeated denoising/model evaluations.

That is the motivation for:

```
diffusion generation
        ↓
        RACS
        ↓
database retrieval
```

---

6. Phase 4 — Build the Retrieval Image Database

Objective

Create the data foundation of RACS.

The overview proposes starting with a manageable image corpus rather than immediately processing a massive dataset.

A practical learning progression is:

```
1,000 images
→ 5,000
→ 10,000
→ larger corpus if required
```

Store

For every image:

```
image_id
image_path
embedding
caption/semantic metadata
structural metadata
```

Important principle

The database becomes a shared protocol resource.

The transmitter and receiver must use:

```
same database version
same preprocessing
same embedding model
same FAISS index
```

Otherwise:

```
Xref_tx != Xref_rx
```

and recovery can fail.

---

7. Phase 5 — CLIP Embeddings

Objective

Understand how semantic similarity becomes a vector.

Conceptually:

```
Image → image embedding
Text  → text embedding
```

Similar semantic content should occupy nearby regions of embedding space.

Build Experiment 1

```
text
 ↓
CLIP text encoder
 ↓
embedding
```

Build Experiment 2

```
image
 ↓
semantic description
 ↓
CLIP text embedding
```

Then compare it against image embeddings.

Critical lesson

CLIP mainly captures semantic similarity.

It does not guarantee:

```
same pose
same layout
same geometry
same texture
same steganographic suitability
```

That limitation motivates structural reranking.

---

8. Phase 6 — FAISS Retrieval

Objective

Build the first real RACS component.

```
Query
 ↓
CLIP embedding
 ↓
FAISS
 ↓
Top-K candidate images
```

Learn

- vector search
- cosine similarity
- inner product
- L2 distance
- nearest neighbours
- approximate nearest-neighbour indexing

Build

Create:

```
build_index.py
retrieve.py
```

Interface:

```
results = retrieve(query_embedding, top_k=10)
```

Test:

```
K = 1
K = 5
K = 10
K = 20
K = 50
```

Save candidate IDs and scores.

Milestone

You now have:

```
Secret image
 ↓
semantic query
 ↓
CLIP
 ↓
FAISS
 ↓
reference candidates
```

This is **RACS-Base retrieval**, before steganography integration.

---

9. Phase 7 — Deterministic Digital-Token Selection

Objective

Make reference selection reproducible and token-controlled.

The token should not directly cause nondeterministic LLM behaviour.

A simple protocol can be:

```
Top-K candidates
      ↓
fixed ordering
      ↓
token/hash
      ↓
deterministic index
      ↓
selected candidate
```

For example:

```
index = deterministic_hash(token) % K
```

The exact mapping should be fixed in configuration and documented.

Test

For the same:

```
token + candidate list
```

the selected candidate must always be identical.

Test:

```
correct token
wrong token
different token
```

Critical concept

Separate:

```
AI reasoning
```

from:

```
protocol-critical deterministic behaviour
```

This becomes especially important when we later add a VLM.

---

10. Phase 8 — Structural Representation

Objective

Address the weakness of semantic-only retrieval.

Example:

```
"person playing football"
```

may retrieve the correct concept but the wrong:

- pose,
- spatial layout,
- object arrangement,
- geometry.

Learn

For general images:

- segmentation
- edges
- depth/layout

For human/facial images:

- pose landmarks
- facial landmarks

The overview explicitly treats semantic information and implicit structural information as separate components.

□filecite□turn14file0□

First build

Do not begin with a complicated learned metric.

Start with:

```
image
 ↓
structural representation
 ↓
structural feature/embedding
```

Then compare secret vs candidate.

---

11. Phase 9 — Structure-Aware Reranking

Objective

Improve the initial CLIP candidates.

Use the conceptual score:

```
Score(I)
=
α × semantic_similarity
+
β × structural_similarity
```

Later, if justified:

```
+ γ × steganographic_suitability
```

Pipeline

```
CLIP
 ↓
Top-20
 ↓
structural scoring
 ↓
rerank
 ↓
best candidate pool
```

Experiment

Compare:

```
A: CLIP only
B: CLIP + structure
```

Do not add VLM yet.

We first need to know whether a simple structural signal already helps.

---

12. Phase 10 — Learn EDICT as a Separate Mini-Project

Objective

EDICT is the hardest new technical area. Do not integrate it immediately with RACS.

Learn:

- diffusion inversion
- deterministic diffusion
- invertibility
- coupled latent variables
- forward/reverse transformations

- numerical reconstruction

Conceptually:

```
latent
 ↓
EDICT forward
 ↓
transformed latent
```

and:

```
transformed latent
 ↓
EDICT inverse
 ↓
original latent
```

First experiment

Do only:

```
latent
 ↓
EDICT forward
 ↓
EDICT inverse
 ↓
latent
```

Measure latent reconstruction error.

Then decode both and compare images.

Completion criterion

You can explain why EDICT is useful for reversible hiding/recovery.

---

13. Phase 11 — Reproduce the Local Steganography Core

Objective

Make the hiding/recovery mechanism work before adding wireless communication.

Conceptually:

```
Secret + Reference
        ↓
       VAE
        ↓
     EDICT
        ↓
     Stego
```

Recovery:

```
Stego
 ↓
VAE
 ↓
EDICT inverse
 ↓
Recovered secret
```

Include the token perturbation mechanism used by the original design.

Measure

```
MSE
PSNR
SSIM
LPIPS
```

Do not add yet

```
AWGN
JSCC
VLM
editing
```

First prove the local reversible core works.

---

14. Phase 12 — Integrate RACS with EDICT

Objective

Now connect your new contribution to the existing steganographic core.

First version:

```
Secret
 ↓
semantic/structural query
 ↓
CLIP
 ↓
FAISS
 ↓
structure reranking
 ↓
token selection
 ↓
Xref
 ↓
EDICT
 ↓
stego
 ↓
EDICT inverse
 ↓
secret
```

This is:

RACS-Base

No VLM. No editing. No generation fallback.

Why?

If it fails, we know the problem is in:

```
retrieval
or
reference compatibility
or
EDICT integration
```

rather than a complicated combination of modules.

---

15. Phase 13 — Add AWGN

Objective

Introduce channel noise only after the local RACS pipeline works.

Learn:

- SNR
- signal power
- noise power
- Gaussian noise

Conceptually:

```
signal + Gaussian noise = received signal
```

Start with:

```
20 dB
```

then:

```
15 dB
10 dB
5 dB
```

The overview uses the 5–20 dB evaluation range. □filecite□turn14file9□

---

16. Phase 14 — Semantic Communication

Objective

Integrate the communication component without turning it into the main research problem.

Conceptually:

```
Stego
 ↓
Semantic Encoder
 ↓
Channel Representation
 ↓
AWGN
 ↓
Semantic Decoder
 ↓
Reconstructed Stego
```

Learn:

- semantic encoder/decoder
- channel representation
- SNR
- bandwidth
- reconstruction

Use a reproducible pretrained/existing implementation where possible instead of training a new communication system from scratch.

The RACS overview explicitly treats semantic communication/channel as components that should initially remain unchanged from the baseline. □filecite□turn14file5□

---

17. Phase 15 — Full RACS-Base

This is the first major milestone.

Transmitter


Secret
↓
Kpub + Xfeat
↓
Query
↓
CLIP
↓
FAISS
↓
Structural reranking
↓
Token selection
↓
Xref
↓
EDICT
↓
Stego
↓
Semantic encoder
↓
AWGN

Receiver

```
AWGN output
 ↓
Semantic decoder
 ↓
Reconstructed stego
 ↓
Same database
 ↓
Same embedding model
 ↓
Same ranking
 ↓
Same token
 ↓
Same Xref
 ↓
EDICT inverse
 ↓
Recovered secret
```

First critical test

Before looking at PSNR:

```
Xref_transmitter == Xref_receiver
```

If this fails, stop and fix reproducibility.

---

18. Phase 16 — Establish the Baseline Comparison

Now build/obtain the most faithful practical AgentSemSteCom baseline possible.

The experimental rule is:

```
same secret images
same image classes
same SNR
same metrics
same downstream pipeline
```

Compare:

```
AgentSemSteCom
vs
RACS-Base
```

Quality

- PSNR
- MSE
- SSIM
- LPIPS

Efficiency

- reference acquisition time
- total latency
- peak GPU memory
- model inference time
- optional FLOPs estimate

Do not claim RACS is faster until measurements demonstrate it.

---

19. Phase 17 — Add VLM / Agentic Reranking

Only after RACS-Base works.

New pipeline

```
Secret
 ↓
FAISS
 ↓
Top-K
 ↓
VLM/Agent
 ↓
candidate evaluation
 ↓
fixed ranking
 ↓
token-controlled selection
 ↓
Xref
```

The VLM can consider:

- semantic compatibility
- layout
- structure
- visual compatibility
- likely editing requirement

Important architecture rule

Do not use a nondeterministic cloud LLM as the critical transmitter/receiver decision mechanism.

Prefer:

```
VLM
 ↓
candidate scores
 ↓
fixed deterministic ranking
 ↓
token
 ↓
selection
```

or a shared frozen/local model.

Compare

```
RACS-Base
vs
RACS-Agent
```

This tells us whether intelligence actually improves retrieval enough to justify its cost.

---

20. Phase 18 — Lightweight Editing

If retrieval finds a candidate that is:

```
semantically good
but structurally imperfect
```

add a lightweight editing stage.

```
Retrieved image
+
Xfeat
 ↓
light image-to-image/structure-guided editing
 ↓
Xref
```

Possible directions from the overview:

- SDEdit-style partial diffusion
- image-to-image adjustment
- structure-guided editing

Critical constraint

Do not turn this into full diffusion generation.

We are trying to reduce the cost of reference generation.

Measure:

```
quality gain
vs
additional latency
```

Compare:

```
RACS-Agent
vs
RACS-AgentEdit
```

---

21. Phase 19 — Retrieve-or-Generate Hybrid

Add a fallback only after retrieval is understood.

```
FAISS
 ↓
candidate quality
 ↓
score >= threshold?
      /           yes       no
     |         |
 retrieve   diffusion
      \       /
        Xref
```

Compare:

```
Always generate
Always retrieve
Retrieve + edit
Retrieve-or-generate
```

This may become an important quality/efficiency result.

---

22. Phase 20 — Security

Test:

Correct token

```
correct token
 ↓
good reference/recovery
```

Wrong token

```
wrong token
 ↓
different selection/perturbation
 ↓
poor recovery
```

No token

Measure degradation.

The original paper reports a substantial correct-token vs wrong-token recovery-quality gap, so RACS should test whether its modified token role preserves a comparable security effect. □filecite□turn7file0□

Do not make security claims from one image. Use a proper test set and report distributions/statistics.

---

23. Phase 21 — Steganalysis

Later, test whether stego images can be detected.

Possible detectors from the research plan:

- XuNet
- optionally SRNet

Basic experiment:

```
reference/cover images
vs
stego images
```

Compare detector performance for:

```
AgentSemSteCom
vs
RACS
```

This should not block the main implementation.

---

24. Phase 22 — Full Evaluation Matrix

Systems

```
AgentSemSteCom
RACS-Base
RACS-Agent
RACS-AgentEdit
RACS-Hybrid
```

SNR

```
5 dB
10 dB
15 dB
20 dB
```

Image classes

Use the same baseline categories wherever practical:

```
common
facial
style
```

Metrics

```
PSNR
MSE
SSIM
LPIPS
Latency
GPU memory
Computation
Security gap
```

---

25. Phase 23 — Ablation Studies

Retrieval

```
CLIP only
vs
CLIP + structure
```

Top-K

```
1
5
10
20
50
```

Token

```
without token selection
vs
with token selection
```

VLM

```
without VLM
vs
VLM reranking
```

Editing

```
retrieval only
vs
retrieval + editing
```

Fallback

```
always retrieve
vs
retrieve-or-generate
```

These experiments are what turn the implementation into a research study rather than merely an engineering replacement.

---

26. Phase 24 — Efficiency Analysis

The main RACS hypothesis is efficiency.

Measure:

```
T_generation
T_retrieval
T_reranking
T_editing
```

and:

```
T_total =
semantic extraction
+
reference acquisition
+
EDICT
+
communication
+
recovery
```

Also measure:

```
peak GPU memory
```

Compare:

```
AgentSemSteCom:
reference = diffusion generation

RACS:
reference = retrieval
             + reranking
             + optional editing
```

The overview's central research question is whether retrieval can provide sufficiently compatible conditioning while reducing computational cost. □filecite□turn14file6□

---

27. Phase 25 — Research Analysis

At the end, answer:

1. Does retrieval reduce reference acquisition cost?
2. How much does PSNR change?
3. How much does SSIM change?
4. How does LPIPS change?
5. Does structural reranking recover quality?
6. Does VLM reranking justify its additional cost?
7. Does lightweight editing improve the quality/latency trade-off?
8. Does the token preserve the security gap?
9. How does RACS behave from 5–20 dB?
10. Which image classes benefit?
11. Where does retrieval fail?
12. Does retrieve-or-generate give the best trade-off?

---

28. Phase 26 — Final Tables and Figures

Quality table

```
System | SNR | PSNR | SSIM | MSE | LPIPS
```

Efficiency table

```
System | Reference Time | Total Time | GPU Memory
```

Security table

```
System | Correct Token | Wrong Token | Gap
```

Ablation table

```
Variant | PSNR | SSIM | LPIPS | Latency
```

Important figures

```
1. AgentSemSteCom vs RACS architecture
2. Reference acquisition latency
3. PSNR vs SNR
4. SSIM vs SNR
5. LPIPS vs SNR
6. Quality vs latency
7. Top-K effect
8. Correct-token vs wrong-token performance
```

---

29. Phase 27 — Reproducibility Package

Freeze:

```
dataset version
database version
FAISS index
embedding model
preprocessing
model checkpoints
random seeds
token mapping
Top-K
ranking weights
editing parameters
EDICT parameters
SNR
test set
```

Every result should be tied to a configuration.

---

30. What You Need to Understand Deeply

You do not need to memorize every implementation detail, but these concepts should eventually become very strong.

1. Why RACS exists

```
Diffusion reference generation
        ↓
high computation/latency
        ↓
Can retrieval provide compatible references more efficiently?
```

2. Why CLIP alone is insufficient

```
semantic similarity != structural compatibility
```

3. Why deterministic selection matters

```
Tx Xref == Rx Xref
```

4. Why EDICT matters

It provides the reversible/invertible diffusion mechanism needed by the hiding/recovery process.

5. Why we change only reference acquisition first

Otherwise we cannot attribute performance changes to RACS.

6. Why VLM comes later

We need a simple retrieval baseline before measuring whether intelligent reranking helps.

7. Why editing must be lightweight

Otherwise we simply recreate the expensive generation problem.

---

31. Your First Implementation Sprint

Do not begin with EDICT.

Start with the easiest components that teach you the architecture.

Sprint 1

Step 1

Set up:

```
Python
PyTorch
CUDA
Git
project structure
```

Step 2

Load and inspect images.

Step 3

Run a pretrained VAE.

Step 4

Run pretrained Stable Diffusion.

Step 5

Measure reference generation time.

Step 6

Load CLIP.

Step 7

Generate embeddings for ~1,000 images.

Step 8

Build a FAISS index.

Step 9

Run:

```
query → Top-10 images
```

Step 10

Implement deterministic token selection.

At the end of Sprint 1, you will already have the fundamental RACS idea working:

```
semantic query
      ↓
vector representation
      ↓
database search
      ↓
candidate selection
```

---

32. How We Should Work Together Phase by Phase

For every phase, follow exactly this loop:

```
1. What are we building?
2. Why does RACS need it?
3. What theory do I need?
4. Learn the theory.
5. Build a tiny experiment.
6. Test it.
7. Understand the output.
8. Integrate it.
9. Save the result.
10. Move to the next phase.
```

For example:

```
CLIP
 ↓
learn embeddings
 ↓
image → embedding
 ↓
compare embeddings
 ↓
FAISS
```

Then:

```
FAISS
 ↓
Top-K retrieval
 ↓
token selection
```

Then:

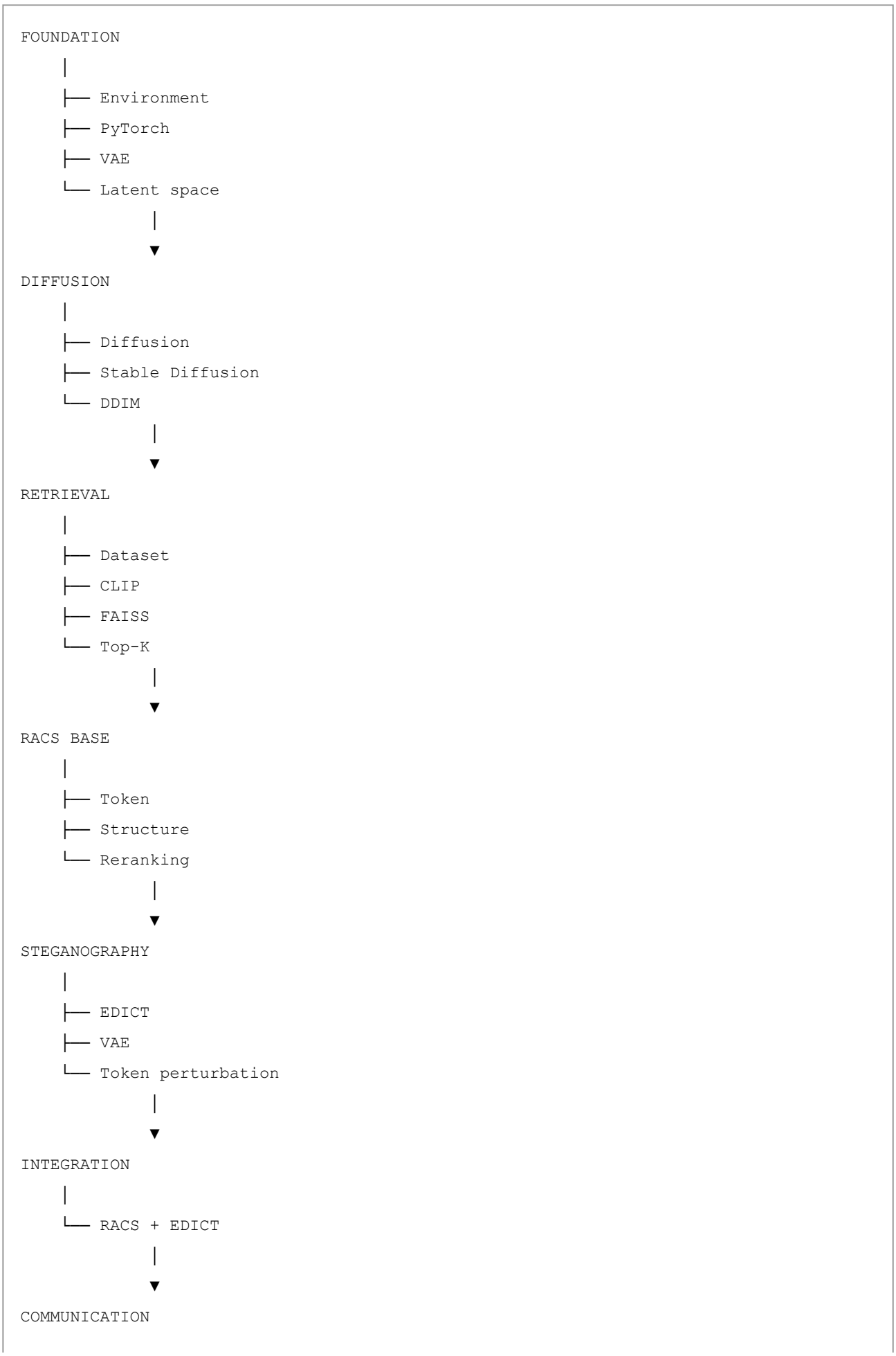
```
structure
 ↓
reranking
```

Then:

```
EDICT
 ↓
integration
```

---

33. Final Roadmap


FOUNDATION
Environment
PyTorch
VAE
Latent space
DIFFUSION
Diffusion
Stable Diffusion
DDIM
RETRIEVAL
Dataset
CLIP
FAISS
Top-K
RACS BASE
Token
Structure
Reranking
STEGANOGRAPHY
EDICT
VAE
Token perturbation
INTEGRATION
RACS + EDICT
COMMUNICATION

```
    |
    ├── AWGN
    └── Semantic communication
              |
              ▼
INTELLIGENT RACS
    |
    ├── VLM
    ├── Editing
    └── Fallback generation
              |
              ▼
EVALUATION
    |
    ├── Quality
    ├── Efficiency
    ├── Security
    ├── Robustness
    └── Steganalysis
              |
              ▼
RESEARCH RESULTS
    |
    ├── Ablations
    ├── Figures
    ├── Tables
    └── Paper
```

---

34. Definition of Project Completion

The project is complete when we can demonstrate:

Functional

```
Secret
 ↓
RACS reference acquisition
 ↓
EDICT
 ↓
Stego
 ↓
Communication
 ↓
Recovery
 ↓
Secret
```

Reproducible

```
Tx reference == Rx reference
```

Comparative

```
AgentSemSteCom
vs
RACS
```

Quantitative

```
PSNR
SSIM
MSE
LPIPS
Latency
Memory
```

Security-aware

```
Correct token
vs
Wrong token
```

Research-aware

```
Top-K
Structure
VLM
Editing
Fallback
```

Honest

We should ultimately be able to say something like:

> *Retrieval reduced reference acquisition latency by X%, while PSNR changed by Y dB, SSIM changed by Z, and the correct-token/wrong-token security gap changed by W dB.*

If retrieval performs worse, that is still a useful scientific result. It can show that semantic retrieval alone is insufficient and identify what structural/agentic components are needed.

---

35. The Most Important Instruction

Do not try to finish RACS in one jump.

We will build it as a sequence of understandable mini-projects:

```
Mini-project 1:
VAE + latent space

Mini-project 2:
Diffusion

Mini-project 3:
CLIP + embeddings

Mini-project 4:
FAISS retrieval

Mini-project 5:
Token selection

Mini-project 6:
Structural reranking

Mini-project 7:
EDICT

Mini-project 8:
RACS + EDICT

Mini-project 9:
AWGN + semantic communication

Mini-project 10:
VLM + editing

Mini-project 11:
Security + experiments
```

The first actual coding target should therefore be **Phase 1** → **Phase 2**, not the complete RACS pipeline.